Report Assignment 4: Finding Shortest Paths using Reinforcement Learning (Dynamic Programming)

Author : Simone Magistrali

Email : smi25011@student.mdu.se

Course : Learning Systems DVA493

Professor : Ning Xiong

Semester : Fall 2025

1. Introduction and Objective

The goal of this assignment is to apply the principles of Reinforcement Learning, and in particular Dynamic Programming, to solve shortest path problems in a graph.

The work is divided into two parts:

1. **Manual Exercise:** Solve a small-scale (5-node) problem via iterative manual computation to demonstrate understanding of the Value Iteration algorithm.
2. **Software Implementation:** Development of a Python program to find optimal routes in a more complex map consisting of 23 cities (file city 1.txt), with the goal of reaching the destination city "F".

2. Methodology: Value Iteration

The method used is the **Value Iteration algorithm** , which is based on the Bellman optimality equation. In this context, the agent (the traveler) must choose the action (moving to a nearby city) that maximizes the sum of expected future rewards.

2.1. Problem Statement

- **States (** S **):** The cities on the map.
- **Actions (** A **):** Move to an adjacent city.
- **Reward (** R **):** Since the goal is to minimize the distance, the reward for each move is defined as the negative value of the distance: R(s, s') = - distance(s, s').
- **Discount factor (** gamma **):** Set to 1.0, since we do not want to discount future costs in a shortest path problem.

2.2. Bellman equation

The algorithm iteratively updates the value V(s) of each city (state) s using the following update rule:

$$V_{k+1}(s) = max_a[\, r_{(s,a)} + V_k(s')\,]$$

Where:

- V_k(s) is the value of city s at iteration k.
- s' is the nearby city reached with the action a.

The process begins by initializing the destination value to 0 and all other values to -inf. Iteration continues until the values converge (i.e., they no longer change between two consecutive iterations).

3. Manual Exercise (Small Problem)

In this section, the problem shown in "Figure 1" of the assignment is manually solved.

- **Nodes:** A, B, C, D, E.
- **Destination:** E.

Initialization V_0:
V(E) = 0; V(A), V(B), V(C), V(D) = -inf

Iteration 1

- $V_1(B) = max\ \ [\, r_{(BE)} + V_0(E),\ r_{(BC)} + V_0(C),\ r_{(BA)} + V_0(A)\,] = max\ \ [-1, -inf, -inf\,] = -1$
- $V_1(C) = max\ \ [\, r_{(CE)} + V_0(E),\ r_{(CB)} + V_0(B),\ r_{(CA)} + V_0(A),\ r_{(CD)} + V_0(D)\,]$

$$= max\ \ [-7, -inf, -inf, -inf\,] = -7$$

- $V_1(D) = max\ \ [\, r_{(DE)} + V_0(E),\ r_{(DC)} + V_0(C),\ r_{(DA)} + V_0(A)\,] = max\ \ [-8, -inf, -inf\,] = -8$
- $V_1(A) = max\ \ [\, r_{(AB)} + V_0(B),\ r_{(AC)} + V_0(C),\ r_{(AD)} + V_0(D)\,] = max\ \ [-inf, -inf, -inf\,] = -inf$

Iteration 2

We update the values using the results of Iteration 1.

- $V_2(B) = max\ \ [\, r_{(BE)} + V_1(E),\ r_{(BC)} + V_1(C),\ r_{(BA)} + V_1(A)\,] = max\ \ [-1, -9, -inf\,] = -1$
- $V_2(C) = max\ \ [\, r_{(CE)} + V_1(E),\ r_{(CB)} + V_1(B),\ r_{(CA)} + V_1(A),\ r_{(CD)} + V_1(D)\,]$

$$= max\ \ [-7, -3, -inf, -12\,] = -3$$

- $V_2(D) = max\ \ [\, r_{(DE)} + V_1(E),\ r_{(DC)} + V_1(C),\ r_{(DA)} + V_1(A)\,] = max\ \ [-8, -10, -inf\,] = -8$
- $V_2(A) = max\ \ [\, r_{(AB)} + V_1(B),\ r_{(AC)} + V_1(C),\ r_{(AD)} + V_1(D)\,] = max\ \ [-3, -10, -12\,] = -3$

Iteration 3

- $V_3(B) = max\ [\, r_{(BE)} + V_2(E),\ r_{(BC)} + V_2(C),\ r_{(BA)} + V_2(A)\,] = max\ [-1, -5, -5\,] = -1$
- $V_3(C) = max\ [\, r_{(CE)} + V_2(E),\ r_{(CB)} + V_2(B),\ r_{(CA)} + V_2(A),\ r_{(CD)} + V_2(D)\,]$

$$= max\ [-7, -3, -6, -12\,] = -3$$

- $V_3(D) = max\ [\, r_{(DE)} + V_2(E),\ r_{(DC)} + V_2(C),\ r_{(DA)} + V_2(A)\,] = max\ [-8, -6, -7\,] = -6$
- $V_3(A) = max\ [\, r_{(AB)} + V_2(B),\ r_{(AC)} + V_2(C),\ r_{(AD)} + V_2(D)\,] = max\ [-3, -6, -12\,] = -3$

Iteration 4 (Convergence)

No further values change. The final optimal values (V*) are:

Node	Optimal Value (V∗)	Real Distance	Optimal Route to E
TO	-3	3	A -> B -> E
B	-1	1	B -> E
C	-3	3	C -> B -> E
D	-6	6	D -> C -> B -> E
AND	0	0	Destination

4. Implementation and Results (23 Cities Map)

A Python program has been developed to solve the larger problem described in the file city 1.txt, with 23 cities and destination **F** .

The program performs the following steps:

1. Loading the map from the text file into an adjacency list.
2. **Value Iteration** loop until convergence.
3. Reconstruction of the optimal route for each city by greedily following the neighbor with the highest R + V(s') value.

Results Achieved

Below are the optimal values and minimum paths calculated by the program to reach the city

F.

Program output:

Calculated Optimal Values (V*)
V*(A): -5
V*(B): -3
V*(C): -5
V*(D): -8
V*(E): -7
V*(F): 0
V*(G): -9
V*(H): -9
V*(I): -13
V*(J): -4
V*(K): -10
V*(L): -2
V*(M): -7
V*(N): -6
V*(O): -7
V*(P): -8
V*(R): -3
V*(S): -7
V*(V): -9
V*(W): -6
V*(X): -6
V*(Y): -1
V*(Z): -8

Actual Minimum Distances (Cost)
Minimum distance from A to F: 5
Minimum distance from B to F: 3
Minimum distance from C to F: 5
Minimum distance from D to F: 8
Minimum distance from E to F: 7
Minimum distance from F to F: -0
Minimum distance from G to F: 9
Minimum distance from H to F: 9
Minimum distance from I to F: 13
Minimum distance from J to F: 4
Minimum distance from K to F: 10
Minimum distance from L to F: 2
Minimum distance from M to F: 7
Minimum distance from N to F: 6
Minimum distance from O to F: 7
Minimum distance from P to F: 8
Minimum distance from R to F: 3

Minimum distance from S to F: 7
Minimum distance from V to F: 9
Minimum distance from W to F: 6
Minimum distance from X to F: 6
Minimum distance from Y to F: 1
Minimum distance from Z to F: 8

Optimal Paths to Goal:
A: A -> B -> F
B: B -> F
C: C -> B -> F
D: D -> B -> F
E: E -> A -> B -> F
F: F
G: G -> J -> L -> F
H: H -> P -> O -> J -> L -> F
I: I -> H -> P -> O -> J -> L -> F
J: J -> L -> F
K: K -> E -> A -> B -> F
L: L -> F
M: M -> F
N: N -> L -> F
O: O -> J -> L -> F
P: P -> O -> J -> L -> F
R: R -> F
S: S -> R -> F
V: V -> S -> R -> F
W: W -> A -> B -> F
X: X -> Y -> F
Y: Y -> F
Z: Z -> X -> Y -> F

5. Conclusion

The implementation of dynamic programming allowed the shortest path problem to be solved. The function V(s) allows the agent to instantly determine the optimal direction at each node, eliminating the need for recalculations.